

Refactoring Summary (Detailed, From Start Of This Chat To Current State)

1) Refactor Objective

The refactor work was driven by two core grading risks:

- `Game` was acting as a god object/service locator.
- MVC boundaries were blurred (model/view/controller responsibilities mixed).

Target architecture used during refactoring:

- Model: gameplay state and rules.
 - View: rendering and UI widgets only.
 - Controller: input routing, screen transitions, side effects (audio, persistence).
-

2) Architecture Before vs After

Before:

- `Game` owned loop, transitions, input facade behavior, side effects, and routing details.
- Input handlers contained state/name-edit/sound branching.
- Views changed game flow directly.
- Rendering concerns were mixed into model entities/level structures.
- Observer/event lists existed without meaningful domain value.

After:

- `Game` is composition root + loop.
 - `GameController` owns state machine, action routing, side effects, render delegation.
 - `KeyboardInputs` is a pure command invoker.
 - Views emit callbacks (`*Actions`), controller states decide transitions.
 - Rendering is in view renderers, not model entities.
 - Direct calls replaced unnecessary observer indirection.
-

3) Concrete Responsibility Moves (What Moved, From Where, To Where)

3.1 God-object split

Moved from:

- `src/main/java/main/controller/Game.java`

Moved to:

- `src/main/java/main/controller/GameController.java`

Now in `GameController`:

- State machine (`GameState`, `setGameState(...)`, current state selection).
- Input action facade implementation (`IGameActions` methods).
- Tick-side effects (`handleModelSideEffects()`).
- Transition scoring hook (`recordLevelCompletion()`).
- Render delegation (`render(...)`, `renderGame(...)`).

`Game` now keeps:

- Bootstrapping/wiring of controller + panel + window.
- Game loop (`run()`), FPS/UPS timing only.

3.2 State classes in controller layer

Controller state files:

- `src/main/java/main/controller/state/GameBaseState.java`
- `src/main/java/main/controller/state/GamingState.java`
- `src/main/java/main/controller/state/MenuState.java`
- `src/main/java/main/controller/state/LeaderboardState.java`
- `src/main/java/main/controller/state/LevelSelectState.java`

Effect:

- Screen behavior and transitions are controller concerns.
- View no longer owns state-machine responsibility.

3.3 View -> callback flow

Action interfaces:

- `src/main/java/main/view/states/Actions/MainMenuActions.java`
- `src/main/java/main/view/states/Actions/LevelSelectActions.java`
- `src/main/java/main/view/states/Actions/LeaderboardActions.java`

Views now call action interfaces:

- `MainMenu.handleSelection(...)` -> `actions.onPlay()` / `onOpenLevelSelect()` / `onOpenLeaderboard()` / `onQuit()`
- `LevelSelect.handleLevelSelection(...)` -> `actions.onSelectLevel(levelIndex)`
- `Leaderboard.backToMenu()` -> `actions.onBackToMenu()`

Controller states implement those callbacks:

- `MenuState` implements `MainMenuActions`
- `LevelSelectState` implements `LevelSelectActions`
- `LeaderboardState` implements `LeaderboardActions`

3.4 Input refactor to command invoker

Invoker:

- `src/main/java/main/controller/inputs/KeyboardInputs.java`

What changed:

- `keyPressed`, `keyReleased`, `keyTyped` only dispatch/execute.
- No game-state checks in key handlers.
- No name-edit branching in key handlers.
- No direct sound logic in key handlers.
- Old `IGameRead` dependency is removed from codebase.

Command files:

- `src/main/java/main/controller/inputs/commands/Command.java`
- `src/main/java/main/controller/inputs/commands/MoveLeftPressCommand.java`
- `src/main/java/main/controller/inputs/commands/MoveLeftReleaseCommand.java`
- `src/main/java/main/controller/inputs/commands/MoveRightPressCommand.java`
- `src/main/java/main/controller/inputs/commands/MoveRightReleaseCommand.java`
- `src/main/java/main/controller/inputs/commands/JumpPressCommand.java`
- `src/main/java/main/controller/inputs/commands/JumpReleaseCommand.java`
- `src/main/java/main/controller/inputs/commands/PlayJumpSoundCommand.java`
- `src/main/java/main/controller/inputs/commands/TogglePauseCommand.java`
- `src/main/java/main/controller/inputs/commands/GoToMenuCommand.java`
- `src/main/java/main/controller/inputs/commands/LeaderboardNextLevelCommand.java`
- `src/main/java/main/controller/inputs/commands/LeaderboardPreviousLevelCommand.java`
- `src/main/java/main/controller/inputs/commands/MenuNameControlKeyCommand.java`
- `src/main/java/main/controller/inputs/commands/MenuNameTypedCommand.java`
- `src/main/java/main/controller/inputs/commands/NoOpCommand.java`

Mouse input path:

- `src/main/java/main/controller/inputs/MouseInputs.java` forwards raw coordinates to `IGameActions`.

3.5 Model rendering concerns moved to view renderers

View renderers now own drawing:

- `src/main/java/main/view/render/LevelRenderer.java`
- `src/main/java/main/view/render/PlayerRenderer.java`
- Orchestrated by `src/main/java/main/view/GameView.java`

Model remains data/logic:

- `src/main/java/main/model/entities/entity/Player.java`
- `src/main/java/main/model/levels/Level.java`
- `src/main/java/main/model/entities/entity/Spike.java`
- `src/main/java/main/model/entities/entity/TriggerPlatform.java`

- `src/main/java/main/model/entities/entity/TriggerSpike.java`
- `src/main/java/main/model/entities/entity/SpawnPlatform.java`
- `src/main/java/main/model/entities/entity/MovingPlatform.java`
- `src/main/java/main/model/entities/entity/DeathSprite.java`

Result:

- No model `Graphics/BufferedImage` rendering methods.
- Model exposes sprite ids/hitboxes/state; renderers map that to images.

3.6 Audio moved out of model domain logic

Audio invocation is in controller/state flow:

- `src/main/java/main/controller/GameController.java`
- `src/main/java/main/controller/state/GamingState.java`
- `src/main/java/main/controller/state/MenuState.java`
- `src/main/java/main/controller/state/LeaderboardState.java`
- `src/main/java/main/controller/state/LevelSelectState.java`

Model no longer decides audio playback directly.

3.7 Jump key repeat fix

Issue:

- Key repeat could fire jump sound repeatedly while jump was held.

Fix:

- `GamingState` owns `jumpSoundArmed`.
- `onPlayJumpSound()` plays only once until `onJumpReleased()` disarms.
- File: `src/main/java/main/controller/state/GamingState.java`

3.8 Observer simplification

Removed:

- Observer-style indirection and listener lists that were adding complexity without clear value.

Current approach:

- Direct controller calls for side effects.
- Controller handles transition-completion effects and scoring.

3.9 Leaderboard view no longer parses file directly

Data model/service:

- `src/main/java/main/model/leaderboard/ScoreEntry.java`
- `src/main/java/main/model/leaderboard/LeaderboardService.java`

View abstraction:

- `Leaderboard.LeaderboardDataSource` in `src/main/java/main/view/states/Leaderboard.java`

Controller wiring:

- `GameController` provides data source implementation to `LeaderboardState`.

3.10 Player/level reload flow deduplication

Centralized helper:

- `GameModel.reloadPlayerForCurrentLevel()`

Used by:

- `GameModel.onEnterMenuFromPlaying()`
- `GameModel.onEnterPlayingFromLevelSelect()`
- `GameController` constructor initial setup
- Transition path in `GameModel.updateTransition()`

Effect:

- Reduced duplicated player-level re-init logic.

3.11 Passive view surface/wiring cleanup

`GamePanel`:

- File: `src/main/java/main/view/GamePanel.java`
- Exposes `attachInputListeners(...)` and delegates rendering to controller.

`GameWindow`:

- File: `src/main/java/main/view/GameWindow.java`
- Focus lost callback routed to panel/controller (`onWindowFocusLost`), not direct game internals.

3.12 Quality gate fix

Build blocker fixed:

- Removed unused import in `src/main/java/main/model/entities/states/PlayerModel.java`.

4) End-to-End Behavior Flows After Refactor

4.1 Jump key while playing

1. `KeyboardInputs.keyPressed(...)` dispatches bound commands for key code.
2. `PlayJumpSoundCommand.execute()` calls `IGameActions.playJumpSound()`.
3. `JumpPressCommand.execute()` calls `IGameActions.jumpPressed()`.
4. `GameController` forwards to current controller state.

5. `GamingState.onPlayJumpSound()` and `GamingState.onJumpPressed()` execute state-specific behavior.

4.2 Main menu button click

1. `MainMenu.mouseReleased(...)` detects selected option.
2. `MainMenu` calls callback, for example `actions.onOpenLevelSelect()`.
3. `MenuState` handles callback and calls `controller.setGameState(LEVEL_SELECT)`.
4. `GameController` performs transition and `onEnter()/onExit()` state hooks.

4.3 Leaderboard navigation from keyboard

1. `KeyboardInputs` dispatches LEFT/RIGHT bound commands.
2. `LeaderboardPreviousLevelCommand` or `LeaderboardNextLevelCommand` executes.
3. `GameController` routes to current state.
4. `LeaderboardState` updates `leaderboardView.previousLevel()/nextLevel()`.

5) Dispatch Logic Description (Current)

The dispatch path in `KeyboardInputs` is now intentionally simple:

- `keyPressed(...)` -> `executePressed(...)` -> execute list from `pressedCommands`.
- `keyReleased(...)` -> `executeReleased(...)` -> execute list from `releasedCommands`.
- `keyTyped(...)` -> `executeTyped(...)` -> execute specific typed command or fallback typed-char command.

No screen checks or branch trees exist in the key event methods.

6) Verification Snapshot

Verified during this refactor stage:

- `mvn -DskipTests compile` passed with checkstyle clean (0 violations at that run).
- No remaining references found for removed observer/read-facade remnants:
 - `GameObserver`
 - `gameEventListeners`
 - `IGameRead`

7) Remaining Strict-MVC Debt (Known, Not Hidden)

The codebase is significantly cleaner, but strict interpretation is not yet fully complete:

- Model still depends on `java.awt.geom.Rectangle2D` for hitboxes:
 - `src/main/java/main/model/entities/entity/Entity.java`
 - `src/main/java/main/model/entities/states/PlayerModel.java`
 - `src/main/java/main/model/entities/states/MovingPlatformModel.java`
 - `src/main/java/main/model/entities/states/TriggerPlatformModel.java`
 - `src/main/java/main/model/entities/states/TriggerSpikeModel.java`

- Model still imports `LoadSave` in:
 - `src/main/java/main/model/levels/LevelManager.java`
 - `src/main/java/main/model/leaderboard/LeaderboardService.java`

These are the main remaining items for very strict MVC/infrastructure decoupling.

8) Final Ownership Map (Current)

- `Game`:
 - composition root
 - game loop
 - input listener wiring
- `GameController`:
 - action facade
 - state machine
 - side effects (audio/persistence triggers)
 - render delegation
- `controller.state.*`:
 - per-screen behavior
 - enter/exit effects
 - screen-specific input handling
- `view.*`:
 - drawing and UI event surfaces
 - callback emission
- `model.*`:
 - player/level/game rules and state progression
 - no direct rendering calls